

DJAP technical manual

How the audio processing works: every stage, its formulas, measurements and sources

Version 0.1.328

https://github.com/hghaken/DJAP

Free software under the GNU General Public License v3

This manual describes how each part of DJAP's audio processing works: the signal flow, the filters and detectors, the formulas, the reasons behind the design, and how it was verified. It is written for audio engineers, developers and curious users. How to use the program is in the user manual (`DJAP-Manual.pdf`, or `docs/manual.md`); the specification with every decision is `docs/design.md`; the code is in `src/core/` (the DSP, no Windows code), `src/app/` (the program) and `src/plugin/` (the plug-in for DJ software, chapter 18) - all on https://github.com/hghaken/DJAP.

Numbers in square brackets, such as [1], refer to the sources in chapter 21, with links. DJAP's code was written for DJAP, with the AI assistant Claude (Anthropic), from these public standards and papers and the project's own measurements; no code, presets or parameter values were taken from commercial products. The one adapted routine is the halfband coefficient designer, which follows the public hiir library (WTFPL licence; [25], `THIRD_PARTY.md`). Where no source is cited (the leveller, the MB Compressor's gain law, the auto tone, the drift loop's tuning), the design is DJAP's own, with its measurements in this manual and in `docs/design.md`.

Contents

1. Overview
2. Building blocks
3. Measuring loudness
4. Leveller; 4a. Noise reduction
5. EQ
6. Compressor
7. MB Compressor (multiband)
8. Bass enhancer
9. Exciter
10. De-esser
11. Stereo width
12. Soft clipper
13. Look-ahead true-peak limiter
14. Output, bypass, fades and the Windows volume
15. Meters and graphs
16. The real-time engine
17. Inputs on another clock
18. The plug-in
19. The loudness presets and the standards behind them
20. Verification
21. Sources

1. Overview

1.1 Signal flow per deck

Each deck is one `DeckProcessor` (`src/core/deck.cpp`). Per sample, in this order:

```
input (stereo)
 -> clean-up: non-finite or absurd samples (|x| > 1e6) -> 0; the input factor of the volume follower
(14.4)
 -> [input meter: K-weighted energy, sample peak]           [dry path: the input delayed by the look-
ahead]
 -> mono (only when chosen): both channels (L + R) / 2, a 20 ms ramp
 -> leveller gain x input gain                              (4)
 -> noise reduction, 4 bands or 1                           (4a)
 -> EQ: sub bass, tone, auto tone, contour                  (5)
 -> compressor                                              (6)
 -> MB Compressor, 1-7 bands                                (7)
 -> bass enhancer -> exciter -> de-esser                    (8-10)
 -> stereo width                                            (11)
 -> [leveller detector]                                     (4)
 -> soft clipper                                            (12)
 -> look-ahead true-peak limiter: the only look-ahead        (13)
 -> output gain (only down) -> bypass crossfade -> clip guard -> fades
 -> [output meter: K-weighted energy, true peak]
 -> Windows volume (only when followed)                     (14)
output
```

Two decks run the same code independently; with ASIO® both are processed in one buffer callback, deck A then deck B, so they share the same latency (they are mixed after DJAP, and a difference would comb-filter).

1.2 Principles

- **Stereo-linked.** Every gain is computed from both channels together and applied to both, so the stereo image never shifts.
- **Double precision.** All filters and gains run in 64-bit floating point; the deck's input and output are 32-bit float. The two channels of a filter run side by side in one SSE2 register (`StereoBiquad`), with the same operations in the same order as the scalar version, so each channel is bit-identical to it.
- **Two rates.** The audio runs per sample; detectors that do not need sample rate (leveller, MB Compressor, de-esser, auto tone) update every 16 samples (the *control block*, 0.33 ms at 48 kHz). The MB Compressor and de-esser gains are ramped linearly across the next block; the leveller's gain is held for the block - it moves at most a few dB per second, so a step is under 0.003 dB.
- **One look-ahead.** Only the limiter looks ahead (default 1.5 ms; 0, 1.5, 2, 3 or 5 ms). No other stage buffers, so the latency the processing reports - and the delay of a click or a drum hit's attack - is exactly the look-ahead. The filters do shift low frequencies a little later than that (their phase; 1.5).
- **Nothing clicks.** Every change while playing is ramped or crossfaded (1.4).
- **Bypass is exact.** The bypass path is the input delayed by the same look-ahead, bit for bit.

1.3 Sample rates

44.1, 48, 88.2 and 96 kHz. Every filter is designed for the actual rate from its analogue prototype; the K-weighting of BS.1770, specified at 48 kHz, is re-derived for other rates (2.5).

1.4 Changes without clicks

A new configuration (a loudness preset, a slider, a stage switch) is computed on the GUI thread and handed to the audio thread (16.2). There:

What changes	How it reaches the sound
a stage switched on or off	a 20 ms linear crossfade between the stage's input and output; a stage that starts from rest starts with a cleared state
bypass	a 20 ms crossfade between the processed and the dry path
input and output gain, compressor make-up, width, bass width, bass enhancer amount, clipper drive	a one-pole glide with a 5 ms time constant
filter frequencies of the compressor side chain, exciter and de-esser	new coefficients, filter memory kept
MB Compressor or noise reduction band count or splits; the width stage's bass split	a second, preallocated filter set is warmed up for 20 ms unheard (a filter started cold on running audio starts with a transient), then the output crossfades to it over 20 ms with a raised cosine (a linear fade's corner was audible on a 120 Hz tone)
bass enhancer frequency	the enhancer fades out (20 ms), takes the new filters, fades back in
EQ filters	coefficients ramped per sample from the old design to the new one (5.1)
look-ahead	fade out 5 ms, change the delay, refill it, fade in 5 ms

A configuration that arrives before the first audio block is applied at once, without ramps.

1.5 Group delay

Filters that split or shape the spectrum - the MB Compressor bands' crossovers, the width stage's bass split, the EQ's shelves, the oversampled clipper - are minimum-phase IIR filters: they add no latency, but they delay low frequencies a little more than high ones (their group delay). Measured beyond the look-ahead at 48 kHz on a -30 dBFS tone (`deck_test`, each tone in its own run; outside review, client 2026-10-26: "write it honestly in the docs"):

Loudness preset	50 Hz	100 Hz	1 kHz	5 kHz
Gentle	2.6 ms	2.8 ms	0.3 ms	0.1 ms
Light	5.2	5.2	0.5	0.2
Medium	11.2	6.1	0.5	0.2
Loud	11.8	6.7	0.9	0.2
Very Loud	13.7	8.8	1.1	0.2
FM Station	14.5	9.8	0.5	0.2
DAB+	0.0	0.0	0.0	0.1
Web Stream	5.2	5.1	0.5	0.1

Most of it comes from the MB Compressor bands (more bands, more delay at the bottom) and the width stage's bass split. A kick's attack and every click still come out after the look-ahead alone; the low end of the same hit follows a few milliseconds later, as it does in an analogue multiband processor. Both decks run the same code, so two decks on the same loudness preset stay aligned; two different presets differ by the difference of their rows. Linear-phase crossovers would remove this but add 10-20 ms of latency to everything - DJAP's latency rule wins.

The oversampled clipper and exciter add their halfband filters' phase on top: about 2.6, 4.0 and 4.75 samples at 2x, 4x and 8x (0.05-0.1 ms at 48 kHz). It is not part of the latency a plug-in reports (an IIR filter's phase, like the crossovers', not a fixed delay); while such a stage fades in or out (20 ms) its delayed and undelayed signal are mixed for that

moment - a comb at about 10 kHz and up for 20 ms, which an outside review measured but which is not audible as such.

2. Building blocks

2.1 Biquads

Second-order IIR sections in transposed direct form II, double precision, with coefficients from Robert Bristow-Johnson's *Audio EQ Cookbook* [8]: low-pass, high-pass and all-pass (Q = 1/sqrt(2), Butterworth), band-pass and high shelf. When coefficients change the filter keeps its memory (only b0..a2 are copied), so a moving filter does not click. Background on the bilinear transform and digital filters: Smith [9].

2.2 Linkwitz-Riley crossovers

Two identical Butterworth sections in a row form a 4th-order Linkwitz-Riley filter (LR4) [16]. Its low-pass and high-pass outputs at the same frequency are in phase with each other and sum to a 2nd-order all-pass: a flat magnitude. Every band split in DJAP (MB Compressor, stereo width, bass enhancer) is built from LR4 pairs.

2.3 State-variable filters

The EQ stage uses state-variable filters with linear trapezoidal integration, after Andrew Simper [15]. Unlike a biquad, their coefficients may change every sample without disturbing the filter's state, so a gain or a frequency can glide smoothly. The shelves and bells match their analogue prototypes within 0.0001 dB.

2.4 Smoothers

- One-pole: `y = target + a (y - target)`, `a = exp(-1 / (T fs))` - reaches 63 % of a step after T.
- Branching smoother (attack / release): a one-pole with one coefficient when the value grows and another when it falls.
- Linear ramp: at most a fixed step per sample or per block (the leveller's dB/s, the 20 ms switches).

2.5 K-weighting and the loudness scale

ITU-R BS.1770 [1] weights each channel with a high shelf (+4 dB above about 1.5 kHz, the head's effect) and a high-pass (the "RLB" curve, about 38 Hz), then takes the mean square summed over the channels. The loudness is `-0.691 + 10 log10(mean square)` in LUFS. The standard gives the coefficients at 48 kHz; DJAP derives them for any rate through the analogue prototype (re-rated at 44.1 kHz, the response matches the 48 kHz one within 0.005 dB to 16 kHz).

DJAP uses this loudness scale for its detectors too: a compressor threshold "relative to the target" then means a loudness, not a peak level.

2.6 True-peak interpolation

The true peak is the peak of the waveform a converter rebuilds between the samples. It is estimated by oversampling with a polyphase windowed-sinc interpolator (BS.1770 Annex 2 [1]). The prototype filter is centred on a tap, so phase 0 is the sample itself and the other phases lie between samples; each new sample reports the largest magnitude over the interval between two samples. The window is a Kaiser window [10]. Two detectors are used:

Use	Oversampling	Taps per phase	Kaiser beta	Delay
Limiter	8x	96	5	48 frames
Output meter	4x	32	7	16 frames

The limiter's detector is long because a short one misses peaks: 32 taps per phase let full-band noise through 1.07 dB over the ceiling; 96 taps hold it. The inner loops use SSE (four phases per sum, both channels per coefficient load).

2.7 Robustness

- Denormal numbers (very small values in decaying filters) are flushed to zero for each block (FTZ / DAZ) and the processor's previous mode is restored afterwards.
- If any filter or detector state becomes non-finite (one NaN from a broken source would otherwise silence the deck for good), the deck resets its state and counts it.
- The audio thread never allocates, locks, waits, does file I/O or throws (16.1).

3. Measuring loudness

The loudness meter (`src/core/loudness.*`) follows ITU-R BS.1770 [1] and EBU Tech 3341 [3] / 3342 [4]:

- The deck sends a record every 10 ms with the K-weighted energy of input and output.
- **Momentary** loudness: the mean over the last 400 ms. **Short-term**: over 3 s.
- **Integrated**: 400 ms blocks every 100 ms (75 % overlap), gated - blocks under -70 LUFS are ignored (absolute gate), then blocks more than 10 LU under the mean of the rest (relative gate).
- **Loudness range (LRA)**: the spread between the 10th and the 95th percentile of the short-term loudness, after an absolute gate at -70 LUFS and a relative gate 20 LU under the mean [4].

The meter is verified with the EBU Tech 3341 test cases (within 0.1 LU) and LRA cases (within 1 LU) at 44.1 and 48 kHz. The offline renders (`DJAPCmd --render`, `--measure`) use the same meter.

4. Leveller

The leveller is a slow automatic gain that brings each song to the loudness preset's target loudness T. It replaces the per-song gain a playback system would compute in advance from an analysis of the file: a live DJ input has no such analysis, so the leveller has to measure as the music plays.

4.1 Detectors

- **Input level** (0.4 s, K-weighted, before any processing): decides whether the music is *quiet* (below `T - Hold below` after the leveller and input gain), *silent* (below -60 LUFS) or playing.
- **Post level**: measured after the stereo width stage - after compressor and MB Compressor have done their work - so the leveller aims the *processed* sound at the target. Three averages, 0.4 s (*fast*), 1.5 s (*mid*) and 3 s (*slow*), and a *trend* level: 0.4 s, but a control block more than 10 dB under the mid level leaves it where it is - the gap between two sung or spoken phrases is not a fall, while every block of a fade passes. It is kept every 0.25 s for 6 s and compared as 1 s averages (the ups and downs of the beat averaged out).
- The leveller's own gain of the last block is divided out of the post level. Without this the 3 s average would lag the leveller's own moves and, with a closed loop through it, a fast catch-up overshot to the end of its range.
- The post level is capped at `T + Range + 6 LU`: a single sample far over full scale (and filters ringing after it) is a glitch, not music, and must not pull the gain down.

4.2 The gain

Every control block, while the music plays and is not quiet:

```
wanted = clamp(T - L_slow, -Range, +Range)        (dB; L_slow = the 3 s post level)
gain   = gain moved towards wanted by at most Speed / control rate
```

Range (default 12 dB) is the largest boost or cut; Speed (default 1 dB/s) limits how fast the gain may move, slow enough not to follow the phrases of a song. Inside the Window (default 1 dB around the wanted gain) it moves at a tenth of that (4.3).

4.3 Rules

- **Hold below.** While the input is more than `Hold below` (10 LU) under the target - a fade, a pause, the start of a quiet passage - the gain is held. A quiet passage that *stays* is music, though: when the music has been under that level for `Quiet wait` (1 s; Gentle and DAB+ 3 s) without falling (the trend level not more than 3.5 dB under its value 4 s earlier) and not more than 12 LU under it (music, not a noise floor), the hold ends and the passage is levelled like any other part.
- **Falling.** The gain may not rise while the music falls: the trend level dropped more than 2.5 dB over the last 2 s (a fade of 1.25 dB/s or more, the drop into a quiet passage), or it sank more than 0.8 dB in each of the last two 2 s (a slow fade: it goes on falling, while a quiet passage is flat again after its drop); a fall counts for at least 1 s. When the music has stopped falling, the 3 s level starts from the 1.5 s level instead of its lag - after a drop of 10 dB a 3 s average needs 6-8 s to come down by itself. In the first moment of a fade, before the trend shows it, the gain also may not rise while the 0.4 s level is more than 3 LU under the 3 s level. Measured: a 40 dB / 10 s fade was lifted 0.7 dB at most, a 40 s fade of 1 dB/s 1.5 dB in its first 4 s (4.7 dB before the trend rule).
- **Warm start.** A 3 s average starting from nothing reads new music as far too quiet for seconds. After a pause (at least 1 s of silence), the slow detector follows the fast one for the first 1.5 s of music, and the gain is held meanwhile (after a start after silence it may come down meanwhile, see below). A quiet passage without silence is not a pause: the leveller goes on from where it was.
- **Fast up.** When the gain is more than 4 dB under what the 3 s level needs and the music is not falling - a quiet verse after a loud chorus, a soft intro - it goes up at the *Release* speed (seconds per 10 dB: 1.5, so 6.7 dB/s; Gentle and DAB+ 5 s, 2 dB/s), until it is within 1 dB. A gap between two phrases does not stop it (only a fall does). The counterpart of fast down. Until 2026-10-27 this was a fixed rule (four times the speed up to 3 dB/s); Release, Attack and Quiet wait are sliders now (client: "a soft intro sometimes takes too long to get to level,

and then it is as if someone turns the volume knob quickly"). Their defaults were chosen on copies of 18 songs with soft intros through Loud: the time until on level summed 322 -> 249 s and the seconds more than 6 LU too soft 601 -> 497, with the steepest rise 6.7 dB/s (1 s per 10 dB took 234 s, but at 10 dB/s) and no song further over the target. Where an intro stayed soft longer, the cause was the Range (an intro more than 12 dB under the target) or a fade-in from silence still more than 12 LU under the hold level (held as a noise floor) - not these speeds.

- **Why these rules** (client, 2026-10-26, a song whose spoken verses lie 10 LU under its choruses): after each chorus the output stayed 6-8 LU under the target for 10-19 s on every loudness preset. The gaps between the phrases had dipped the 0.4 s level under the hold level and under the old fall rule (no move for half of each verse, and a 1.5 s warm restart after every gap of 1 s), the 3 s level needed 6-8 s to come down to the verse, and the normal speed another 7 s. A synthetic verse 10 LU under its chorus with a 0.8 s gap every 3 s stayed at -24.1 LUFS (target -14) with the old rules and is levelled to -14.0 LUFS from 9 s on now; the drop itself still sounds (the first 2 s are not lifted), as do the first 3 s of a breakdown.
- **Catch-up.** After at least 2 s of silence (a stopped deck) the next song is caught up - up at the Release speed, down at the Attack speed (until 2026-10-27 a switch, at 20 dB/s both ways) - following the 0.4 s level. Catch-up goes on while the 0.4 s level still rises more than 1 dB per second (a track fading in), for at most 15 s; then the 3 s detector starts from the level reached. Measured: a 12 s fade-in from -50 dB ended 5.1 LU too loud without, 0.04 LU with.
- **Start after silence.** After at least 2 s in which every control block is under -60 LUFS (a stopped deck) the gain goes to *After silence* (+12 dB by default, at most the Range), whatever it was, as a 60 dB/s ramp inside the silence (client, 2026-11-08; a slider since 2026-11-12, "hold" leaves the gain where it was). A soft intro is on level from its first moment. A louder song starts the catch-up at once, without the 1.5 s of the warm start, but in those 1.5 s the gain only comes down: the 0.4 s level, rising from nothing, can only read too low (from the top the gain can only come down anyway). A softer song goes on up after them. The silence also clears the trend detector: it kept the last song's level, and a soft song after a loud one read as a fall (no turning up for 6 s). The price is a loud song's first moments: at -8 LUFS after 3 s of silence in Medium (target -14) the output is -7 .. -9 LUFS for the first second and on target after about 3 s; the Attack sets how long that lasts. Freeze keeps its gain, and a stream that starts in the music starts as before: only measured silence counts.
- **Fast down (Attack).** When the gain is more than 3 dB above what the 3 s level needs - a loud song after a quiet intro or a break - it comes down at the *Attack* speed (seconds per 10 dB, default 1 s: 10 dB/s), following the 0.4 s level (the 3 s average lags a loud entry), only down, until it is within 0.5 dB; then the 3 s level starts from the level reached. Because it is judged on the 3 s level, a chorus 4 LU above the verse does not trigger it. Measured on a 16 dB entry after a 6 s intro: 4-10 s after the entry the output was -6.0 LUFS without fast down and -17.9 LUFS with it (target -18).
- **Fast down behind the compressor.** The detector sits after the compressor and the MB compressor (4.1), and they hide most of a loud entry from it: with a 2.5:1 compressor an entry 14 dB up reads as 5-6 dB, and as the gain comes down at the normal speed the "3 dB too loud" of fast down is never reached. Measured with Loud's compressor: the output stayed 4 LU over the target for 4 s, with 6-7 dB of compression all that time, where the leveller alone was on the target in 2 s. So "clearly too loud" is also judged at the leveller's own output. While the gain is within 1 dB of what the detector asks, the leveller learns the chain's usual gain: the detector's 3 s level over the K-weighted 3 s level of the input (10 s average, 3 s before it counts, learned again after every change of a setting). The level that goes with the target at the leveller's output is then the target minus that gain, and fast down starts when the input's 3 s level with the leveller's gain is more than 3 dB above it while the detector reads too loud as well (a stage that turns a quiet passage down, like the noise reduction, is no loud entry). It then follows the input's 0.4 s level, stops 1 dB before that level's place - the usual gain differs a little from song to song - and the rest goes at the normal speed on the detector's own reading. With the compressor on, the 14 dB entry is now within 2 LU of the target after 1.2 s (Loud; before: 7.8 s) and 1.4 s (Medium; before: 4.3 s).
- **Window.** A song's 3 s level wanders a decibel up and down from phrase to phrase, and a gain that follows all of it never stands still: on the first 150 s of 25 songs it travelled about 30 dB per minute of music. While the gain is within the *Window* (default 1 dB) of what the 3 s level needs, it rests - it only drifts, at a tenth of the Speed, so the level still ends on the target. When the level leaves the window the gain moves at the Speed again, all the way, and rests when it has arrived. Fast up, fast down and the catch-up are not slowed, and for 3 s after them the gain does not rest yet (the 3 s level starts again there). 0 dB switches the window off.

Measured on those songs: the travel halves (32.8 -> 16.6 dB per minute in Medium) and the output's short-term spread grows by about 0.5 LU (3.5 -> 4.1 LU); a window of 2 dB gains little more (15.0 dB per minute, 4.5 LU).

4.4 Why the detector sits after the dynamics stages

A leveller that measures before the compressor and MB Compressor aims the *unprocessed* level at the target; the stages after it then change the loudness again, and the output misses the target by however much they compress or lift. Measuring after them (the handbook's "measure what you control") puts the output on target for every loudness preset.

4a. Noise reduction

A downward expander in four bands or one (`NoiseBank` in `src/core/deck.cpp`; client, 2026-10-29). It takes hiss, hum and rumble away where the music does not cover them - pauses, fades, quiet passages - and leaves music above its threshold untouched. It sits right after the leveller and input gain, so its threshold can be relative to the target like the compressor's and the de-esser's (the leveller has brought every song to the same level), and before the EQ, so a treble boost does not lift the hiss into its detector. Off in every shipped loudness preset.

1. **Bands.** Three LR4 splits (default 80 Hz, 1 kHz, 5 kHz: rumble and hum, low, mid, hiss), built like the MB Compressor bank (7.1): the two lower bands pass the all-passes of the splits above them, so the four bands sum to an all-pass. Or one band: the gain on the whole signal, no filters.
2. **Level.** Per band the mean square of both channels, averaged with a one-pole over one period of the band's lowest frequency (at least 5 ms; the lowest band and the wide band from 30 Hz: 33 ms), on the loudness scale. A shorter average let a bass tone's own ripple open and close its band; a detector that jumped to every 16-sample block's peak read noise 6-8 dB too loud.
3. **Gain computer.** With `u` = how far the level is under the threshold (target + Threshold setting, -80..-20 LU, default -45) and the ratio 1:R, the reduction is `(R - 1) u`, with a 6 dB quadratic knee around the threshold as the compressor's (6), at most Depth (0..40 dB, default 12).
4. **Times.** The reduction is smoothed in dB once per control block: it shrinks (the band opens) with the Attack time (0.5..50 ms, default 2); before it grows, the band waits Hold (0..500 ms, default 50) - started again by every control block in which the band asks to open - and then follows the Release time (20..2000 ms, default 250). The gain is ramped linearly across the next control block. A band that is open again within 0.0001 dB is set to exactly 0 dB, so a wide band at rest passes the signal bit for bit.
5. **Changes.** A new band count or new splits run in the spare bank with the 20 ms warm-up and 20 ms raised-cosine crossfade of the MB Compressor bank (7.4); the spare bank starts from the running bank's levels and reductions (from 1 band to 4 every band gets the wide band's, from 4 to 1 the wide band gets the least reduced band's), so the crossfade does not swell.

It has no noise-floor estimate and no signal-to-noise ratio: the user's fixed threshold decides, per band. Within about an octave of a split a loud band leaks into its neighbour through the LR4 slopes and holds it open (a 2 kHz tone at -20 dBFS reaches the band above 5 kHz at about -53), as music does: the hiss band closes under quiet music, not under a full mix. Its splits add their phase (1.5) only while the stage is on. Measured (`noise_test`): open, the four bands sum flat within 0.0001 dB from 40 Hz to 12 kHz; pink noise 19 dB under the threshold comes down by the depth (-11.98 dB at 12, -29.94 at 30); a 500 Hz tone at -20 dBFS with hiss under it keeps its level within 0.005 dB while the hiss above 8 kHz goes down 18.8 dB (one band: the tone holds it open, the hiss stays); after a drop the band waits the detector's fall and the hold, then reaches 63 % of the depth one release time later; no change of any value clicks (worst click ratio 1.00). Cost with four bands: about 0.16 % of one core per deck (`deck_test --cpu`); nothing while it is off.

5. EQ

The EQ stage (`src/core/colour.*`) changes the tone of the music and nothing else. It sits after the leveller and before the leveller's detector, so the leveller keeps the loudness on target: an EQ changes the balance, not the volume.

5.1 Tone

Four sections, each a state-variable filter (2.3):

Section	Type	Gain	Frequency
Bass	low shelf, Q 0.71	-12..+12 dB	40..250 Hz (100)
Low-mid	bell, Q 0.9	-12..+12 dB	150..1000 Hz (350)
Presence	bell, Q 0.9	-12..+12 dB	1000..6000 Hz (3000)
Treble	high shelf, Q 0.71	-12..+12 dB	3000..16000 Hz (8000)

Targets glide with a 25 ms one-pole. Each section is redesigned once per control block (16 samples) when it moved by at least 0.005 dB or 0.05 Hz, and its coefficients are ramped linearly per sample from the old design to the new one. Stepping them per block instead produced a measurable zipper (a click ratio of 98.9 on a 60 Hz tone; with the ramp below 1). A section that has arrived at 0 dB is skipped and its state cleared, so a neutral EQ is bit-exact.

5.2 Auto tone

Pulls each song's spectral balance towards a reference:

1. Five band-passes (Q 1) at 60, 250, 1000, 3000 and 10000 Hz measure the deck's input (mono, after the input gain) and keep a long-term power average per band (time 2..60 s, default 10 s; 1.5 s during the first 6 s of a song so a new song is corrected soon).
2. The *balance* of a band is its level against the 1 kHz band.
3. The correction of the bass, low-mid, presence and treble sections is `clamp((reference - balance) x amount, +-maximum)`, added to the section's own gain. The treble gets half the maximum: the top of an old recording is often its hiss.
4. The averages and the correction hold while the deck is quiet or falling (the leveller's rules), so a breakdown or a fade does not turn the EQ.

The reference is the median balance of a set of late-1980s dance and pop productions: +5.9 / +3.4 / 0 / -1.5 / -5.2 dB at 60 / 250 / 1000 / 3000 / 10000 Hz. `DJAPCmd --tone` prints the balance of any file, so the reference can be recalibrated on other music. Measured corrections at 100 %: a 1956 rock'n'roll single +6 dB bass / +3 dB treble; a late-80s dance track -2.3 / +1.2 dB.

5.3 Sub bass

Adds the octave below the bass, which many records of the 1960s and 1970s lack:

1. The deck's bass below the sub frequency (40..120 Hz) is isolated with a low-pass (two SVF sections).
2. It drives a flip-flop that toggles on every rising zero crossing (with 10 % envelope hysteresis against noise): a square wave at half the frequency - an octave lower.
3. The square wave is multiplied by the bass's envelope and low-passed at 0.6 x the sub frequency (two sections), so only its fundamental remains.

Measured: a 60 Hz tone gets 30 Hz at -2.3 dB (amount 100 %) or -8.3 dB (50 %); silence stays silent. This is the classic octave-divider principle of analogue sub-harmonic synthesisers, decades old; the implementation is DJAP's own (no source is cited for it).

5.4 Contour

At a low listening volume the ear is less sensitive to low and high frequencies (the equal-loudness contours of ISO 226 [17]). The contour's amount adds a low shelf at 100 Hz (up to +9 dB) and a high shelf at 10 kHz (up to +4 dB), measured +8.9 dB at 30 Hz and +3.9 dB at 16 kHz at 100 %.

6. Compressor

A wide-band, feed-forward compressor with a soft knee, after the design analysis of Giannoulis, Massberg and Reiss [6].

1. **Side chain.** The signal after leveller, input gain and EQ, through a 2nd-order high-pass (default 150 Hz, 0 = off, up to 200 Hz) so the bass does not trigger the compressor. A wide-band compressor that follows the kick turns everything else down with every beat - the "pumping" of a compressor. With the filter at 80 Hz (the default until 2026-11-12) a kick at -1 dBFS over a steady pad pulled the pad's highs down 2.0 dB per beat in Medium, 3.5 dB in Loud, 6.3 dB in Very Loud and 8.0 dB in FM Station; at 150 Hz that is 0.3, 0.9, 2.2 and 3.3 dB, with the same output loudness (what is left comes from the MB compressor and the clipper, which work per band and on the peaks themselves).

2. **Level.** The summed square of both channels, smoothed with a 10 ms one-pole, on the loudness scale: `x = -0.691 + 10 log10(L^2 + R^2)`. A per-sample peak would compress every transient; a short mean square on the loudness scale makes the threshold mean a loudness.

3. **Gain computer** - the static curve in dB, threshold t = T + threshold setting, ratio R, knee width W:

```
x - t < -W/2:          y = x
|x - t| <= W/2:       y = x + (1/R - 1) (x - t + W/2)^2 / (2W)
x - t > W/2:           y = t + (x - t) / R
```

4. **Smoothing.** The gain reduction `x - y` is smoothed in dB *after* the gain computer, with the attack time when it grows and the release time when it shrinks (the "smooth, decoupled" detector of [6]).

5. **Make-up.** Automatic: the curve's reduction at the target level, so material at the target passes unchanged and only louder material is compressed. Or a fixed value, 0..12 dB.

Measured step response with the defaults: 63 % of the reduction after 14 ms (attack 10 ms plus the 10 ms detector), back to 37 % after 223 ms.

7. MB Compressor (multiband)

The MB Compressor compares each frequency band with **its own recent level** rather than with a fixed threshold. Material above that level is turned down, material below it is lifted: soft details come up, peaks are held, and a bass drum moves only the low band. This is what makes the dense, even sound of broadcast processing.

7.1 The band split

A cascade of LR4 splits, each taking its band off the rest of the signal (lowest band first). Every lower band then passes the 2nd-order all-passes of the splits above its own, so all bands share the same phase and sum to an all-pass: a flat magnitude (measured within 0.01 dB for every band count, and 0.001 dB from 30 Hz to 15 kHz with five bands).

```
rest = input
for each split s:  band[s] = LR4 low-pass(rest);  rest = LR4 high-pass(rest)
band[last] = rest
for each band b:   pass band[b] through the all-passes of splits b+1 .. last
```

With K splits this costs K(K-1)/2 all-passes per channel: 7 bands = 6 splits = 12 LR4 filters (24 biquads) and 15 all-passes, 39 biquads per channel. Both channels run in one SSE2 register.

Default splits per band count (each split movable, always rising and at least 1.25x apart):

Bands	Splits (Hz)
1	- (wide-band)
2	200
3	200 / 2500
4	120 / 600 / 4000
5	60 / 200 / 2500 / 6000
6	60 / 200 / 800 / 2500 / 6000
7	60 / 150 / 400 / 1000 / 2500 / 6000

7.2 Per band (both channels together, every control block)

- **Level**: the band's mean square, smoothed over 10 ms.
- **Reference**: the band's level averaged over 3 s - learnt only while the music is not quiet (the leveller's quiet rule, judged on the *incoming* level). It is kept from falling more than 30 dB under the level itself.
- **Excess** = level - reference, in dB. With `k = 1 - 1/ratio` and `ratio = 1 + 3 x Amount` (1:1 at 0 %, 4:1 at 100 %):
  - above `Down above` (3 dB): turned down by `k x (excess - DownAbove)`, at most 24 dB;
  - below `-Lift below` (6 dB): lifted by `k x (-LiftBelow - excess)`, at most `Maximum lift` - never while quiet and never more than 40 dB under the reference.
- The gain moves with the attack time (20 ms) when it goes down and the release time (300 ms) when it goes up, and is ramped linearly across each control block.
- A fixed **band gain** (-12..+12 dB, glided) multiplies on top: with Amount 0 % the stage is a plain multiband tone control.

7.3 Two rules found by testing

1. The quiet rule must look at the *incoming* level, never at the stage's own output: a band that was turned down would otherwise look quiet, stop learning and stay down.
2. The reference must not start empty: an empty reference makes all music look far "above" it and turns it down by up to 12 dB for 1-2 s. For the first 0.5 s of music the reference is learnt fast (0.3 s) and the gain stays at 0 dB.

7.4 Changing the bands while playing

A new band count or new splits change the filters' phase, so they cannot simply be swapped. Each deck holds two complete filter banks (7 bands each, allocated in advance); the new configuration runs in the spare bank, warms up 20 ms unheard and is crossfaded in over 20 ms with a raised cosine. A split change keeps the band levels and gains. Measured cost: the MB Compressor with 7 bands about 0.3-0.4 % of one CPU core per deck.

8. Bass enhancer

Small speakers cannot reproduce the lowest bass, but the ear reconstructs a pitch from its overtones (the "missing fundamental"). Adding overtones of the bass makes it audible on small speakers without adding low-frequency energy (Larsen and Aarts [12], [13]).

1. The mono bass below the enhancer frequency (60..160 Hz, default 100 Hz) is isolated with an LR4 low-pass.
2. It is full-wave rectified, `|x|`: that produces the even overtones 2f, 4f, ...
3. The result is band-passed from f to 4f (2nd-order high-pass and low-pass) and added to both channels with gain `2.4 x Amount` (a rectified sine's 2nd harmonic is 0.42 of the sine; 2.4 brings it to about the same strength). The bass itself is not raised.

Test: 50 Hz gets a 100 Hz overtone 9.4 dB under it at 30 %; off adds nothing; 1 kHz passes unchanged.

9. Exciter

Adds overtones to the highs - shine and air rather than more of the same treble:

1. The highs above the exciter frequency (1500..8000 Hz, default 3500) are isolated with a 4th-order high-pass.
2. They pass a soft saturation `tanh(4x) / 4`. Only **what the saturation adds** is used: `x - tanh(4x)/4` - odd overtones, more on loud highs than on soft ones.
3. That residue is high-passed again (4th order) and mixed in by the amount.

The saturation runs at twice the sample rate (the oversampler of 12.1): the overtones of highs reach far above the Nyquist frequency (the 3rd of 10 kHz is 30 kHz) and, at the base rate, folded back into the audio band as inharmonic tones.

Test: 5 kHz gets 15 kHz 27 dB under it; 500 Hz passes unchanged. Saturation-based enhancement of this kind is described in the DAFX book's chapter on nonlinear processing [7].

10. De-esser

Turns sharp "s", "sh" and "t" sounds down when they stand out:

1. **Detector**: the band above the de-esser frequency (3000..10000 Hz, default 5500) through a 4th-order high-pass, both channels, 2 ms mean square, on the loudness scale.
2. Above the threshold (LU relative to the target, default -10): reduction 4:1 (`0.75 x excess`), at most the maximum (default 6 dB), attack 1 ms, release 80 ms.
3. The reduction is applied as a high shelf at the de-esser frequency (RBJ [8]), recomputed every control block. At 0 dB no filter runs at all (exactly transparent); the shelf's state is cleared when it engages.

Test: a loud 7 kHz tone is turned down 4.5 dB; a 1 kHz tone at the same level is not.

11. Stereo width

Mid / side processing, the principle of Blumlein's 1931 stereo patent [18]:

```
M = (L + R) / 2,   S = (L - R) / 2
S' = BassWidth x LR4low(S) + Width x LR4high(S)       (split at the bass split, 40..400 Hz)
M' = allpass(M)                                        (the same all-pass the LR4 pair has)
L = M' + S',   R = M' - S'
```

- Width 0..200 % (100 % = as recorded, 0 % = mono); bass width 0..100 % (0 % = bass in mono, tight and the same on every speaker; widening the bass is not offered).
- The side is split with an **in-phase LR4 pair** and the mid passes the matching all-pass, so left and right stay in phase with each other. An earlier variant, "side minus its high-pass", is not a clean low band: at 0 % width 4 % of the stereo remained.
- A mono signal stays mono and keeps its level; at 100 / 100 % every frequency keeps its level (only the all-pass phase changes).
- The stage sits before the limiter, because a wider image can mean higher peaks.

Test: a tone on the left channel only (amplitude 1) comes out at 200 % width as L 1.5 and R -0.5, at 0 % as L 0.5 and R 0.5 (mono).

12. Soft clipper

Rounds off the highest peaks without delay, before the limiter, so the limiter has less to do. The signal, scaled to the ceiling (u = 1 is the ceiling) after the drive (0..9 dB), passes a curve that is straight up to half the ceiling and then bends with tanh towards the ceiling - never past it, and without a corner:

```
|u| <= 0.5:   y = u
|u| >  0.5:   y = sign(u) x (0.5 + 0.5 tanh((|u| - 0.5) / 0.5))
```

- The drive makes the music louder and denser by up to its amount; quiet music only gets the gain. It is not compensated (like an input gain).
- **2x, 4x or 8x oversampling** (since 2026-10-26; before, it ran at the base rate; the factor is the user's choice since 2026-10-27, 4x by default - client: "CPU is not a problem"). An outside review measured the aliases of 10-15 kHz tones at -16..-44 dBc at 48 kHz: a curve without a corner keeps them low for midrange tones (a 3 kHz tone driven 6 dB in: 42 dB under it) but not for the highs - hi-hats and cymbals in hard-driven material. First-order antiderivative anti-aliasing was rejected (it acts as a two-sample average: up to 4 dB less at 15 kHz).

12.1 The oversampler

`StereoOversampler` (`src/core/dsp.h`): up to 2, 4 or 8 times the rate, the curve on every sample, back down - a cascade of 2x stages. The first stage's halfband filters are polyphase IIR filters - two chains of first-order allpasses in z^2 [25] - with 8 coefficients from the closed-form elliptic design (transition band 0.02 of the doubled rate): images and aliases 82 dB down, flat within 0.00001 dB to 0.46 x the sample rate (20.3 kHz at 44.1 kHz). Up and down together delay low frequencies by about 2.6 samples (0.05 ms at 48 kHz) and add no latency - a linear-phase FIR halfband of the same quality would add tens of samples. The ceiling enters the curve per doubled sample (`ceiling x curve(u / ceiling)`) and the filters see only the driven signal, so a change of the ceiling meets no samples of the old scale in their memory (a click). The later stages (2x <-> 4x, 4x <-> 8x) see a signal that is band-limited already: 6 coefficients with a transition of 0.11, 108 dB down, about 0.85 base samples of delay each. Measured (`deck_test`, tanh(3x) at 48 kHz): a 13 kHz tone's alias at 9 kHz from -13.2 dBc at the base rate to -97.4 dBc at 2x; a 15 kHz tone's fold-back at 21 / 9 kHz -23.3 / -33.0 dBc at 2x (its 5th and 7th overtones fold inside the doubled band), -113.5 / -110.2 dBc at 4x, -120.3 / -118.1 dBc at 8x; 1 kHz passes 4x and 8x unchanged (0.00000 dB). A new factor while playing: the clipper and the exciter fade out over 20 ms, take it with clean filters and fade back in (the factors delay the signal differently: a switch would click; `deck_test`: click ratio 1.02 for 4x -> 8x -> 2x). Cost per deck at 4x: 0.31 % of a core for the clipper, 0.57 % for the exciter.

At twice the rate the curve never passes the ceiling, but back at the base rate a flattened peak, its overtones above the audio band removed, rises over it again: 0.36 dB for a 3 kHz tone driven 6 dB in, more for the highs (a band-limited square wave overshoots). The limiter after the clipper holds the ceiling, as it does in every loudness preset; holding it in the clipper instead (a hard clamp) brought the fold-back back (-24 dBc on 13 kHz). Through clipper and limiter, a 13 kHz tone driven 6 dB in at 48 kHz: its fold-back at 9 kHz 94.6 dB under the tone, the peak at -1.2 dBFS (`dynamics_test`). The hardest case, tones of 12-16 kHz driven 9 dB in at -6 dBFS: at 2x a rest of 39.8 dB under the tone at 15 kHz (at the base rate: about 16 dB under), at the default 4x 62 dB under in the whole chain. With the limiter switched off in a loudness preset, the clipper holds the ceiling itself (a hard hold after the down filter), and its curve then bends 1.5 dB under the ceiling, so that the hold seldom has a peak to cut: 10 and 15 kHz tones driven 9 dB in at -6 dBFS fold back 59.7 dB under the tone (with the curve at the ceiling: about 25 dB under - fourth outside review; 1 dB under gave about 37), the sample peak stays at the ceiling. The price: the clipped peaks come out about 1.5 dB lower. The curve's ceiling glides (5 ms) when the limiter is switched. The sample peak is all the clipper can hold: the true peak between the samples can rise 2-5 dB over the ceiling on noise - for a true-peak ceiling keep the limiter on. The exciter's saturation: a 10 kHz tone at -6 dBFS, 100 %, its fold-back at 18 kHz 90.2 dB under the tone (20.6 dB at the base rate).

- The meter shows how far the clipper took the deepest peak down.

13. Look-ahead true-peak limiter

The safety net: the output never passes the ceiling (default -1.0 dBTP, -6..0). Look-ahead limiting is described in the DAFX book [7]; DJAP's version (`src/core/limiter.h`) works per sample:

1. **Wanted gain** `w = min(1, ceiling / peak)`, with the peak from the 8x true-peak detector (2.6) - or the sample peak when the look-ahead is shorter than the detector's delay.
2. **Sliding minimum** of w over a window of W samples (a monotonic deque in fixed arrays: O(1) amortised, no allocation - the streaming minimum filter of [27]).
3. **Moving average** of those minima over the same W samples: the gain reaches its value *before* the peak arrives, along a smooth curve without a step.
4. **Release**: when the average is below the current gain it is taken at once; otherwise the gain rises with a one-pole release (default 80 ms, 10..500 ms).
5. The audio is **delayed** by the look-ahead and multiplied by the gain.

The exact window. The gain applied to the sample leaving the delay line must never be above that sample's own wanted gain. With a sample-peak detector this gives `W = look + 1`; with a true-peak detector of delay D (48 frames here) the wanted gain of a sample is known D frames after it arrives (from the reports of the intervals on both sides of it), so `W = look - D + 1`, and the true peak is used only when `look >= D`. A shorter look-ahead fell back to the sample peak without a word - true peaks up to 3 dB over the ceiling (outside review, 2026-10-26) - so a look-ahead above 0 is now never shorter than D + 18 = 66 frames (`lookFrames`, a window of at least 19 frames - the window of the default at 44.1 kHz: 1.50 ms at 44.1 kHz, 1.38 ms at 48 kHz; a window of 16 let full-band noise bursts 0.10-0.11 dB over the ceiling, past the margin, a window of 1 0.18 dB); the choices are 0 (sample peak, no delay), 1.5, 2, 3 and 5 ms, and an old 0.5 or 1.0 ms becomes 1.5. The plug-in reports the same frames to its host. The default 1.5 ms is 66 frames at 44.1 kHz and 72 at 48 kHz. A first version was one or two samples off: a single peak went up to 0.4 dB over the ceiling.

Margin. In true-peak mode the limiter aims 0.1 dB under the ceiling: with a 1.5 ms look-ahead its gain changes fast enough to create small inter-sample peaks of its own (full-band noise went 0.17 dB over without the margin).

Guarantee. The sample peak never passes the ceiling. The true peak stays within the ceiling + 0.1 dB for material below about 0.48 x the sample rate (all music in practice), with any look-ahead above 0. The detector rolls off above that: an outside review measured, against an ideal 16x reference, +0.25..+0.46 dB on clipped pink noise, +0.5..+0.7 dB on full-band noise and up to +1.4 dB on a sine at 0.495 x the rate. The soft clipper before the limiter does not change that much: a fourth review measured +0.2..+0.3 dB at 0.48-0.495 x the rate with the clipper on, and up to +1.6 dB in a loudness preset without it - a limit of the detector's look-ahead (a longer detector would cost latency), met only by test tones just under the Nyquist frequency. At 0 ms the limiter holds the sample peak only. Tests: single +12 dBFS samples at look-aheads of 1.5, 0.2, 0.1 and 0 ms; a +6 dBFS sine; an fs/4 square wave (large inter-sample peaks); +9.5 dBFS full-band noise bursts - all measured with a 16x reference true-peak meter.

The moving-average accumulator is re-summed exactly every 8192 samples to stop floating-point drift.

14. Output, bypass, fades and the Windows volume

14.1 Output gain and clip guard

The output gain (-15..0 dB) only turns down, so the ceiling holds. A clip guard then replaces non-finite values with 0 and clamps to [-1, 1]; it never acts on a valid signal, and it is skipped in full bypass so bypass stays bit-exact.

14.1a Dither

An ASIO output of 16, 18 or 20 bits gets TPDF dither (two uniform +-0.5 steps, so +-1 step triangular) before the rounding: plain rounding leaves distortion that follows the music at low levels (a fade, a reverb tail), dither a steady faint noise instead (about -98 dBFS at 16 bits) [26]. A value already on a step gets none, so digital silence stays silent and a bypassed 16-bit input leaves bit-exact. 24 bits and more are only rounded: that rounding lies under any converter's own noise. Measured (`engine_test`): a tone of 0.4 steps, lost to rounding, comes out at 0.399 steps, without an offset; the error variance is 1/4 step^2 for any value (TPDF; rectangular dither would give 1/6). A block of the device period that stays under a quarter step everywhere gets no dither: the faint remains of a filter or a release after the music round to digital silence instead of a lasting +-1 step of noise (the tail of +-1 step after the music lasts a few ms to some tens of ms, by rate and block size - at about -96 dBFS). Below a quarter step the output is silence (about -102 dBFS rms at 16 bits); above it a steady tone is dithered (a 0.9-step tone: 3rd harmonic -47 dBc; with the old limit of one step it came out undithered at -22 dBc, and a 0.4-step tone vanished - fourth outside review, measured).

14.2 Bypass

The dry path is the input delayed by the same look-ahead, so processed and dry are aligned in time and the 20 ms bypass crossfade never jumps. Full bypass is the input, bit for bit.

14.3 Fades at start and stop

- Every live start fades the deck's output in over 5 ms, so a stream started in the middle of music does not begin with a step (a hard click). Offline renders and tests start bit-exact without a fade.
- Before a running stream stops - Stop, a new device, Shutdown, Windows or a setup closing the program - each deck fades to silence over 10 ms (raised cosine) and stays silent; the program waits until every running deck reports silence (at most 300 ms), then 50 ms more so the silence passes the device's buffers.

14.4 Following the Windows volume

Windows applies its volume (keyboard keys, taskbar) only to streams into its default playback device - not to ASIO and not to DJAP's outputs - and if it turned down a virtual cable DJAP reads, the leveller would turn it back up. So DJAP follows that volume itself, when the option is on:

- The program reads the default playback device's volume (in dB, as Windows or the driver applies it, at most 0 dB) and mute every 15 ms.
- Each deck multiplies its output by it after the clip guard (only down, so the ceiling holds; volume 1 is bit-exact), with a 20 ms glide. The meters show the level before it.
- When the default playback device is the deck's own input (a virtual cable) and Windows applies the volume in software, the deck's input is multiplied by the inverse (at most +40 dB), so the processing sees the music's own level and the volume counts once.
- That factor cannot move at the very moment the music with the new volume arrives: the music is still on its way through Windows' mixer, the cable and the input's buffers (30-60 ms with WASAPI). After the volume is turned down, the chain therefore gets the old level with the new, higher factor for that long - 12 dB too much after a jump from 100 % to 25 %. For 100 ms after the factor was raised the compressor, the MB compressor and the de-esser keep the gain they have, and the leveller's detectors leave those blocks out. Measured with the factor 40 ms before the music: without this the output sagged 4.9 dB after the jump and needed 0.45 s to come back; with it 0.9 dB.

15. Meters and graphs

15.1 Meter records

Every 10 ms each deck writes a record into a lock-free ring (16.2): the K-weighted energy of input and output, the sample peak of the input, the true peak of the output (4x detector), the gain of every stage (leveller gain; compressor, de-esser and limiter reduction; the noise reduction's largest reduction per band; MB Compressor gain per band; how far the clipper reduced the deepest peak), and the energy at 13 points of the chain (`MeterRecord::chain`). The GUI reads the ring 20 times a second.

15.2 Signal-chain bars

For the analysis window the GUI averages the chain energies over 0.4 s and shows, per step, the level after the step against the level before it. The noise reduction (its most reduced band), compressor, de-esser, clipper and limiter show their own gain reduction instead, because their level change hides it (make-up gain, the clipper's drive). The bars use a square-root scale so 0.5 dB is visible and 12 dB fills the bar.

15.3 Spectrum

Each deck also copies every input and output frame into a second ring (four floats per frame - the only extra work on the audio thread). On the GUI thread: a 4096-point radix-2 FFT [19] with a Hann window [20], a new frame every 2048 samples, power averaged with a time constant of 0.3 s (the spectrum window's *Averaging*: none, 0.1, 0.2, 0.3, 0.5, 1 or 2 s - an exponential average of the power per bin, so a sudden drop shows slowly in dB: 20 dB down reads only about 3 dB lower a quarter of a second later at 0.3 s), shown at 256 log-spaced points from 20 Hz to 20 kHz (the loudest bin in each point's band, interpolated where the bins are wider than the points). A sine of amplitude A reads 20 log10(A) dB.

15.4 Stereo statistics

Averaged over 0.3 s on the output (and the input for the width):

```
correlation = sum(L R) / sqrt(sum(L^2) sum(R^2))      (+1 mono, 0 unrelated, -1 opposite)
width       = sqrt(sum(S^2) / sum(M^2)) x 100 %       (M = (L+R)/2, S = (L-R)/2; 0 % = mono)
```

The goniometer plots the output's last 43 ms with mono vertical and left / right on the diagonals.

The scope shows the input or the output (a right-click chooses) as 1, 5, 10, 20, 50 or 200 ms of the samples themselves - the picture ends at the newest rising zero crossing of the left channel, so a steady tone stands still - or as the last 2 s in 400 buckets of each channel's lowest and highest sample. It counts the samples at full scale (|x| >= 0.9999, -0.001 dBFS) over the last 2 s whatever the span shown. At the output only the clip guard puts samples there - it holds anything over full scale - so a red bucket is clipping that left the processor; at the input it is clipping in the source. `analysis_test`: a sine 25 % over full scale, clamped, counts 41 % of its samples, a clean one none.

16. The real-time engine

16.1 Rules for the audio thread

The audio callback has a hard deadline (a period of a few milliseconds); missing it is an audible dropout. So the audio thread never allocates memory, takes a blocking lock, does file or network I/O, or throws. All buffers are allocated before the stream starts; the deck's test counts allocations during processing, parameter changes and switches (0).

16.2 Passing data between threads

- **Settings to the audio thread**: the GUI computes a complete coefficient set per deck (`DeckConfig`) and publishes it in a sequence-locked slot (`SeqLockSlot`): the writer increments a counter before and after writing; the audio thread copies the slot and accepts the copy only if the counter was even and unchanged. It never waits.
- **Values from the audio thread**: single-producer / single-consumer rings (`SpscRing`) for the meter records and the analysis frames; a full ring drops data instead of waiting.

16.3 The ASIO buffer switch

With ASIO, the driver calls DJAP once per period with all channels (`AudioEngine::process`): DJAP converts the input buffers from the driver's sample format (all little-endian PCM and float formats), processes deck A then deck B in chunks of at most 256 frames through fixed buffers, converts into the output buffers (clamped, rounded) and signals the driver. Only the channels in use get buffers. Driver reset requests and buffer-size changes are debounced for 500 ms and then handled on the GUI thread by closing and reopening the driver (ASIO SDK [21]): a running stream fades out first, and when the driver is not ready yet (a USB interface that enumerates again, a new rate set in its control panel) it is tried again every second for a minute; the status bar says so meanwhile.

The other APIs (WASAPI, DirectSound, MME) run through miniaudio [22]. With Drift correction (Auto, the default; for WASAPI not for an input and output of one device - Always: also those) each deck is a capture stream from its input device into its drift buffer (`AsyncInput`, chapter 17, as for an ASIO output) and a playback stream that reads it, processes and plays; without it, one duplex stream from its input device to its output device - no buffer, but two devices on different clocks drift apart and give a glitch now and then. Each stream runs on its own thread, registered with Windows' multimedia scheduler (MMCSS) as "Pro Audio" for WASAPI. In shared mode, when the rate asked for is the device's own, miniaudio uses Windows' low-latency audio engine (IAudioClient3) by itself; in exclusive mode the stream has 2 periods instead of 3, one period less latency. DJAP also asks Windows not to throttle it for power (EcoQoS): it usually sits hidden in the notification area, where Windows 11 may otherwise slow a program down and coalesce its timers.

16.4 Health

Per period the engine measures its own processing time against the period's length: the heaviest and the last period in %, the periods over half their time, the late periods (dropouts) and the periods answered with silence.

16.5 Cost

Both decks with the full chain at 48 kHz use about 4.3-5.9 % of one CPU core at the default 4x oversampling, depending on the machine and its load (target under 3 %; the test fails above 7 % - client, 2026-10-27: "CPU is not a problem"). The deck is bound by latency, not by arithmetic: each sample runs one long chain of dependent operations through all stages (a biquad's feedback is about 12 CPU cycles), so faster maths did not help; processing both channels in one SSE2 register and using fewer serial filters did.

17. Inputs on another clock

With ASIO output, a deck's input may come from a WASAPI, DirectSound or MME device - for example a player sending into a virtual cable; with those APIs as output too, Drift correction (Auto by default) puts the same buffer between a deck's input and output device (16.3). The two devices run on their own crystals, so their sample rates differ by tens of ppm (typically up to +-100 ppm); a buffer read at the nominal rate would run dry or overflow within minutes. With ASIO output the input's period counts twice: once as the input's own latency and once in the drift buffer (which holds a whole input block). Shared WASAPI hands over 10 ms by default; Setup's *Input period* (8, 6, 5 or 4 ms) asks Windows' low-latency shared mode (IAudioClient3) for that period, put on the device's step between its shortest and longest (a request off the step is rounded down under itself and falls back to 10 ms). The mode needs a device that has it, running at the same rate in Windows as the ASIO interface. A request is never put on a step longer than Windows' default period (the default is used instead). The input latency shown is the measured steady block size - the median finished burst of the last 20-40 s, which neither a stall's catch-up nor one short burst moves (the drift buffer holds the largest) - not the device's reported period; a device that hands over longer blocks than its period (a virtual cable's 10.67 ms) shows them as they are. Health compares it with the period on the device's step and with the device's default, and names the cause: no step fits, another rate in Windows, the device kept its default, or its own step. Blocks that repeat the one before bit for bit (16 frames or more) are counted when 1-3 repeats are followed by a new block: the device was read faster than it was filled, and Health says so from three in the last 20-40 s on. A flat block (DC, a clipped plateau) and a longer run of equal blocks (a test tone with a whole number of periods per block) are not counted. A virtual cable fills its buffer on a timer of its own: measured with a 1 kHz tone through one, the period must span 3-4 of its ticks - with a 3 ms timer clean from 8 ms, with 1 ms from 4 ms; shorter periods clicked or read the same data twice.

The input's format. The sample rate sets the highest frequency that can pass, half the rate (Nyquist): 22.05 kHz at 44.1 kHz, 24 kHz at 48 kHz - both above hearing, so the rate itself makes no audible difference. A *change* of rate does: each resampler on the way filters and interpolates, and a virtual cable's own one is linear (it dulls the top octave and lets images through). Hence one rate from the source to DJAP's output, the music's own (manual 3.5); DJAP's oversampling of the clipper and exciter (2x-8x, IIR halfbands) makes a higher overall rate unnecessary. The bit depth sets the quantisation noise: a full-scale sine against the rounding error gives 6.02 N + 1.76 dB - about 98 dB at 16 bits and 146 dB at 24 bits (converters reach 110-120 dB in practice). The floor matters where gain is taken back later: the Windows volume turns the sound down before the cable, the volume follower undoes that at DJAP's input (5.2 of the manual), and the leveller adds up to +12 dB. A cut of 20 dB undone and +12 dB on top lift a 16-bit floor from about -98 to about -66 dB relative to full scale - audible in fades and breaks; at 24 bits it stays near -114 dB. The cable carries fixed-point formats only (no float), so 24 bits; DJAP asks WASAPI for 32-bit float, which holds 24 bits without loss.

`AsyncInput` (`src/core/asyncinput.*`) solves the drift:

- **Ring.** The capture stream writes stereo blocks with a time stamp into a lock-free ring of 32768 frames.
- **Interpolator.** The ASIO side reads it through a 32-tap windowed-sinc interpolator (Kaiser beta 8, cut-off at Nyquist, 256 tabulated fractional positions with linear interpolation between them) - band-limited interpolation as described by Smith [11]. Position 0 is a unit impulse, so at a read speed of exactly 1 the output is the input, bit-exact. Measured: flat within 0.001 dB to 18 kHz, residual below -90 dB to 10 kHz.
- **Drift loop.** The read speed is 1 plus a PI controller on the buffer's fill, at most +-0.2 % (3.5 cents, inaudible) - a second-order loop of the kind [28] describes for filtering audio time stamps. The fill is estimated at the moment of reading (the last block's end plus the time since it came), so the sawtooth of the capture blocks does not modulate the speed; smoothed over 0.5 s, the error corrected over 4 s, the drift estimate following over 16 s (critically damped). With simulated clocks at -500..+500 ppm and late callbacks: drift found within 2 ppm, fill within 0.5 ms of its target, read-speed wobble about 15 ppm, no xruns.
- **Target fill** = the largest capture block + the output period + half the interpolator + a 3 ms margin: about 19 ms for 10 ms capture blocks and 256-frame periods at 48 kHz. "Block" and "period" are bursts: callbacks that come within half the last one's length after it count as one (a device period cut into short callbacks arrives at once; counting only the callback gave underruns after each start). The output's callbacks are merged only from its first real gap on, and reading starts after its second (a stream fills its buffer at the start with callbacks

straight after each other, and that is no burst; seventh review: whether the first capture block came before or after that must not matter; and an output period cut in two is counted whole before reading starts - at the cost of one to two output periods more at the start: eighth review, +2.7 ms at 64 frames, +10 ms at 480, +43 ms at 2048, about nothing in the usual start where reading waits for the fill anyway). Callbacks before the first gap are not remembered at all (a DirectSound stream's first callback is its whole buffer, and counted it held the target 20 ms higher for 40-65 s); an output that never pauses starts reading after 1 s anyway. The largest burst of the last 20-40 s counts, at most 50 ms or 1.5 device periods, whichever is more: the catch-up after a stall is a burst too, and it must not raise the buffer for good (a fifth outside review measured 125 ms after a 100 ms stall). The target goes up at once but down by at most 0.3 ms per second - a read speed of 300 ppm (0.5 cent): a forgotten burst dropped it 40 ms at once and the read speed jumped to +2000 ppm, a pitch step (sixth review). So after a 100 ms stall the burst stays counted for 20-40 s and then comes down at 0.3 ms per second: the target is within 1 ms of its base after about 3 minutes (seventh and eighth reviews: 65 ms until 25-60 s, about 38 ms at 120 s, 29 ms at 150 s, 24 ms at 200 s), and the 2 ms of margin the stall's underrun added decay after that (0.5 ms per quiet minute). The price of that: a stall of 70-80 ms or more that comes back every minute or so gives a dropout each time, where a buffer that only grew would have swallowed the later ones. The drift estimate learns only while the read speed is inside its limits (anti-windup): a fill far over the target ran it into the limit, and it then overshot into an underrun. Each underrun adds 2 ms of margin (up to 20 ms), and every minute without one takes 0.5 ms back down to the 3 ms, so one stall does not keep the delay up (the read speed glides to the smaller fill: no gap; simulated, a 200 ms capture stall was back at the original 18.69 ms within 400 s). An underrun right after such a step down sets a floor at the margin that held: with lasting jitter larger than the base margin a plain decay walked back into the failing margin every ~4 minutes (an outside review measured 15 underruns an hour); simulated with 8 ms of capture jitter: 4 underruns while the margin finds its level, then none in 40 minutes. The deck fades in over 10 ms whenever reading (re)starts, and at an underrun it fades out over 3 ms: what is still in the buffer, then its last frame held (before, the music stopped with a step - a click; with nearly nothing left the step came back - sixth review). Measured at 30 moments each at output periods of 64, 128 and 480 frames: the largest step from one frame to the next is the sine's own.

- **Same path for WASAPI / DirectSound / MME decks** with Drift correction: a capture stream writes, a playback stream reads. On *Auto* (the default) WASAPI leaves an input and output of one device (the same container id, not the computer's own) one duplex stream: one clock, nothing to correct - saving the buffer minus the period that duplex keeps, 5-13 ms at 64-480 frames. *Always* corrects them too, for a device that holds two clocks after all; *Off* never corrects.

This is the only place where DJAP adds buffering beyond the look-ahead, and only when the user chooses such an input. The Setup window shows the added latency and the measured drift.

18. The plug-in

The plug-in (`src/plugin/`, built with Steinberg's plug-in SDK [24]) runs one `DeckProcessor` inside a host such as DJ software. Everything in chapters 2-15 applies unchanged: the plug-in calls the same deck on the host's buffers.

- **Audio.** Stereo in, stereo out, 32-bit float, the host's sample rate and block size (any size, also 1 frame). The host's buffers are processed in place by `DeckProcessor::process`. Activation (outside the audio thread) prepares the deck - the only allocations - and starts the 5 ms fade-in of a live start.
- **One object.** Processor and controller are one object (the SDK's single-component effect), so the plug-in's window reads the deck's meter and tap rings itself through the same `DeckMonitor` (`src/core/monitor.*`) the program uses: one reader per ring, as the rings require (16.2).
- **Parameters.** The 83 values of the chain plus the loudness preset and EQ lists, Bypass and the look-ahead. A value's normalised position (0..1) maps to the program's slider positions (`paramPositions`, `paramValueAt`, `paramPositionOf`): linear in the value's steps, logarithmic (300 positions) for times and frequencies spanning a factor of 8 or more. A parameter's id is an FNV-1a hash of its settings key plus 1000, so ids stay the same when values are added or reordered; a test checks that every id is unique.
- **Changes in the audio thread.** Parameter changes arrive with each block; the processor takes the block's last value of each, and when anything changed designs a new `DeckConfig` (`makeDeckConfig`: coefficients only, no allocation) and hands it to the deck through its sequence-locked slot before processing the block. The deck then ramps or crossfades as in 1.4: no automation step clicks.
- **Loudness presets.** Choosing a loudness preset or EQ (the list parameters) makes the controller load that loudness preset's values, ask the host to read all parameters again (`kParamValuesChanged`) and hand the processor a complete snapshot; the processor never reads the loudness preset lists, which change on the GUI thread. The controller keeps the exact values of a loudness preset or a project, so a stored project holds 800 Hz and not the 810 Hz of the nearest log step; a value the host sets is that step's value.
- **Shared loudness presets.** The plug-in's loudness window is the program's (`editwnd.*`). Program and plug-ins save into one settings file, each with a three-way merge (`Settings::saveMerged`): the file is read again, the writer's own changes since it last read or wrote the file (its "base") are applied to it - additions, changed values, renames, deletions - and everything else stays. Two writers therefore never lose each other's loudness presets; when both changed the same loudness preset, the later save wins. A plug-in writes only the loudness preset and EQ lists, never the program's other settings, and nothing when it has nothing new. Each writer saves through a temporary file of its own name and renames it over the file, so a reader never sees half a file.
- **Latency.** The look-ahead in samples (`round(ms x rate / 1000)`, 72 at 1.5 ms and 48 kHz) is reported to the host, and announced again (`kLatencyChanged`) when the look-ahead or the sample rate changes, so the host can delay the other channels by the same amount (plug-in delay compensation).
- **State.** A versioned binary stream: loudness preset and EQ names, look-ahead, Bypass and every value as a pair of settings key and value, the oversampling, mono, and the loudness preset's and EQ's values the project's values were based on. Loading merges three-way: a value the project left as its preset had it takes the preset's value in the settings file now (the preset may have been changed since), a value changed in the project stays. Loading it sets the controller and hands the processor a complete snapshot through a second sequence-locked slot (the host may load a project while audio runs).
- **Window.** A Win32 child window in the host's window, scaled to its DPI, built from the program's own parts: the brushed metal (`theme.*`), the meter panel (`meterview.*`), the loudness window and tooltips. A DLL must not leave window classes registered after it is unloaded (they would point into freed code), so the classes and GDI+ are registered with the first open window and removed with the last. Common controls version 6 come from the DLL's own manifest (`ISOLATION_AWARE_ENABLED`), whatever the host's manifest says.
- **Verification.** `plugin_test` loads the built plug-in like a host and compares it with a `DeckProcessor` given the same values: bit-exact over 8 s at block sizes 512, 333, 1024, 64 and 1 with a target change half-way; bit-exact after a state is stored and loaded into a new instance; Bypass is the input delayed by the reported latency, bit-exact. The SDK's validator passes all 47 standard and 537 extended tests (bus arrangements, sample rates, block sizes, parameter and state handling, threading rules).

19. The loudness presets and the standards behind them

The loudness presets' targets follow the loudness standards of the media they imitate:

- **EBU R 128** [2] introduced loudness normalisation in broadcasting: programme loudness -23 LUFS, true peak at most -1 dBTP. **EBU Tech 3344** [5] applies it to distribution and reproduction. The **DAB+** loudness preset follows it: -23 LUFS, wide dynamics, songs shifted slowly instead of squashed, compression only above -18 LUFS (R 128's short-term maximum for "s1"), no MB Compressor, clipping or exciter (a low-bit-rate codec smears clipping and spends its bits on added highs).
- **FM radio** is limited by the multiplex power of ITU-R BS.412 [14] (an average over 60 s), which caps the loudness at about -12..-15 LUFS while station processing makes the sound very dense. The **FM Station** loudness preset processes like a loud station (-9 LUFS) and turns the result down 4 dB, leaving peaks only about 8 dB above the loudness (measured -13.3 LUFS, -5.1 dBTP). It is the sound, not a transmitter-ready signal: no 15 kHz low-pass and no stereo coder.
- **Streaming and file playback**: the AES recommendations for audio streaming and network file playback (TD1004, then TD1008, later the AES77 standard) [23] put music at about -16 LUFS for portable listening; music services normalise to -14..-16 LUFS. The **Web Stream** loudness preset uses -16 LUFS without clipping and with a ceiling of -2 dBTP, because lossy coding raises peaks slightly and turns clipping into audible distortion. **Medium** sits at -14 LUFS.
- The five strength loudness presets (Gentle -18 .. Very Loud -10 LUFS) step about 2 LU at a time; each compresses a little earlier and harder and uses more bands and more density. Gentle sits at the low end of the AES playback range (-16..-20 LUFS); Very Loud approaches modern club masters (-8..-9 LUFS) but keeps a margin so the limiter does not carry everything, and keeps its release at 50 ms or more so a 50 Hz bass is not distorted by the limiter's gain following the waveform.

The reasons for every value are in `docs/design.md` sections 3 and 3.1.

20. Verification

Each area has its own test program (`tests/`, run all with `tools\run_tests.ps1 -Renders`):

Area	Test	Pass
Loudness	EBU Tech 3341 cases and LRA cases (Tech 3342) at 44.1 / 48 kHz; silence	+-0.1 LU; LRA +-1 LU; -inf
K-weighting	44.1 kHz response against 48 kHz	0.005 dB to 16 kHz
True peak	Tech 3341 true-peak cases	+0.2 / -0.4 dB
Compressor	static curve; step response	exact; 63 % in about 14 ms
Limiter	+12 dBFS single samples at several look-aheads; +6 dBFS sine; fs/4 square; +9.5 dBFS noise bursts	sample peak <= ceiling; true peak (16x reference) <= ceiling + 0.1 dB
Bypass	processing off	the input delayed by the look-ahead, bit-exact
MB Compressor	band-sum flatness for every band count 1..7	0.01 dB
Noise reduction	band sum while open; pink noise under the threshold; a tone with hiss under it (4 bands and 1); hold / release / attack; every value changed on a tone it turns down; the split rule	0.001 dB (one band bit-exact); the depth within 0.3 dB; the tone within 0.05 dB, the hiss down by about the depth; the times; no click; rising, 1.25x apart
Band count	3 -> 7 -> 1 bands on a sine while playing	no step larger than the signal's own
Clicks	every parameter changed on a sine in its own band	no step larger than the signal's own
EQ	every section against its analogue prototype; contour; neutral; auto tone; sub bass	0.02 dB; +8.9 / +3.9 dB; bit-exact; reference - balance; 30 Hz -2.3 dB
Width	left-only tone at 200 % and 0 %; mono stays mono	1.5 / 0.5; 0.5 / 0.5
Loudness presets	all 8 loudness presets on test material at -18 and -9 LUFS in	output within 1 LU of each loudness preset's target; true peak <= ceiling
Robustness	NaN / Inf input; FTZ restored	recovers within a block; processor mode unchanged
Real time	allocation counter during processing and changes	0 allocations
Decks	deck A's processing does not change deck B; equal latency	bit-exact; the same delay
CPU	both decks, full chain, 48 kHz	< 3 % of one core is the goal (fail > 7 %); measured 4.3-5.9 % at 4x oversampling
Inputs on another clock	simulated clocks -500..+500 ppm, late callbacks, pauses	no xruns, drift within 3 ppm, fill within 0.5 ms
Graphs	FFT against a DFT; sines 50 Hz - 18 kHz; correlation and width of known signals	1e-12; within a bin; +-0.05 / 1 %
Offline	synthetic songs through every loudness preset, checked with an independent meter	integrated loudness within 1 LU of the target, true peak <= ceiling
Plug-in	loaded like a host: against the deck at block sizes 1..1024 with a change; state stored and loaded; Bypass; latency; parameter ids; the SDK's validator	bit-exact; 83 of 83 values, bit-exact; bit-exact; 72 / 144 samples; unique; 47 + 537 tests passed
Slider positions	every value's positions: range, order, every shipped value back within a step	exact ends, rising, within a step (2 % on log scales)

The offline render check runs the real deck code on synthetic songs (fades, pauses, loud entries) and measures the results with a separate, independently written loudness and true-peak meter.

21. Sources

Standards, papers and books the design is based on. All links were checked on 2026-10-26; some publishers' pages need a membership or a purchase for the full text.

#	Source	Used for
[1]	ITU-R BS.1770-5 (2023), *Algorithms to measure audio programme loudness and true-peak audio level* - https://www.itu.int/rec/R-REC-BS.1770-5-202311-I	K-weighting, gating, true peak (2.5, 2.6, 3)
[2]	EBU R 128, *Loudness normalisation and permitted maximum level of audio signals* - https://tech.ebu.ch/publications/r128	loudness normalisation, LUFS / LU, -1 dBTP (19)
[3]	EBU Tech 3341, *Loudness Metering: "EBU Mode" metering to supplement EBU R 128* - https://tech.ebu.ch/publications/tech3341	momentary / short-term / integrated, test cases (3, 19)
[4]	EBU Tech 3342, *Loudness Range: a measure to supplement EBU R 128* - https://tech.ebu.ch/publications/tech3342	loudness range (3)
[5]	EBU Tech 3344, *Guidelines for distribution and reproduction in accordance with EBU R 128* - https://tech.ebu.ch/publications/tech3344	the DAB+ loudness preset (19)
[6]	D. Giannoulis, M. Massberg, J. D. Reiss, *Digital Dynamic Range Compressor Design - A Tutorial and Analysis*, J. Audio Eng. Soc. 60(6), 2012, pp. 399-408 - https://aes2.org/publications/elibrary-page/?id=16354	compressor structure, soft knee, smoothing (6)
[7]	U. Zoelzer (ed.), *DAFX - Digital Audio Effects*, 2nd ed., Wiley 2011 - https://dafx.de/DAFX_Book_Page_2nd_edition/	look-ahead limiting, nonlinear processing (9, 13)
[8]	R. Bristow-Johnson, *Audio EQ Cookbook* (W3C note) - https://www.w3.org/TR/audio-eq-cookbook/	biquads, shelves (2.1, 10)
[9]	J. O. Smith III, *Introduction to Digital Filters*, CCRMA, Stanford - https://ccrma.stanford.edu/~jos/filters/	bilinear transform, filter structures (2.1)
[10]	J. O. Smith III, *Spectral Audio Signal Processing*: "The Kaiser Window" - https://ccrma.stanford.edu/~jos/sasp/Kaiser_Window.html; after J. F. Kaiser, *Nonrecursive digital filter design using the I0-sinh window function*, Proc. IEEE ISCAS 1974, pp. 20-23	windows of the true-peak and interpolation filters (2.6, 17)
[11]	J. O. Smith III, *Digital Audio Resampling Home Page* - https://ccrma.stanford.edu/~jos/resample/	band-limited interpolation (17)
[12]	E. Larsen, R. M. Aarts, *Reproducing Low-Pitched Signals through Small Loudspeakers*, J. Audio Eng. Soc. 50(3), 2002, pp. 147-164 - https://aes2.org/publications/elibrary-page/?id=11088; summary at https://research.tue.nl/en/publications/reproducing-low-pitched-signals-through-small-loudspeakers/	bass enhancer (8)
[13]	E. Larsen, R. M. Aarts, *Audio Bandwidth Extension: Application of Psychoacoustics, Signal Processing and Loudspeaker Design*, Wiley 2004	bass enhancer (8)
[14]	ITU-R BS.412-9, *Planning standards for terrestrial FM sound broadcasting at VHF* - https://www.itu.int/rec/R-REC-BS.412/en	multiplex power limit, the FM Station loudness preset (19)
[15]	A. Simper, *Linear Trap Optimised SVF* (Cytomic technical paper) - https://cytomic.com/files/dsp/SvfLinearTrapOptimised2.pdf	state-variable filters of the EQ stage (2.3, 5)
[16]	S. H. Linkwitz, *Active Crossover Networks for Noncoincident Drivers*, J. Audio Eng. Soc. 24(1), 1976, pp. 2-8 - https://aes2.org/publications/elibrary-page/?id=2649; see also https://www.linkwitzlab.com/filters.htm	Linkwitz-Riley band splits (2.2, 7, 11)
[17]	ISO 226:2023, *Acoustics - Normal equal-loudness-level contours* - https://www.iso.org/standard/83117.html	contour (5.4)
[18]	A. D. Blumlein, British patent 394,325 (filed 1931, granted 1933), *Improvements in and relating to sound-transmission, sound-recording and sound-reproducing systems*	mid / side stereo (11)

#	Source	Used for
[19]	J. W. Cooley, J. W. Tukey, *An algorithm for the machine calculation of complex Fourier series*, Mathematics of Computation 19 (1965) - https://doi.org/10.1090/S0025-5718-1965-0178586-1	FFT (15.3)
[20]	F. J. Harris, *On the use of windows for harmonic analysis with the discrete Fourier transform*, Proc. IEEE 66(1), 1978 - https://doi.org/10.1109/PROC.1978.10837	Hann window (15.3)
[21]	Steinberg, ASIO SDK (used under its GPL-3.0 option) - https://www.steinberg.net/developers/	ASIO driver interface (16.3)
[22]	D. Reid, miniaudio (public domain / MIT-0) - https://miniaud.io/	WASAPI, DirectSound and MME streams (16.3)
[23]	AES Technical Council, TD1004 (2015) and TD1008 (2021), *Recommendations for loudness of internet audio streaming and on-demand distribution* (TD1008 later became the AES77 standard) - AES technical documents: https://aes2.org/?p=50870	streaming targets, the Web Stream loudness preset (19)
[24]	Steinberg Media Technologies, plug-in SDK v3.8.1 (MIT licence; the plug-in interfaces, base classes, validator and test hosts) - https://github.com/steinbergmedia/vst3sdk	the plug-in (18)
[25]	R. A. Valenzuela, A. G. Constantinides, *Digital signal processing schemes for efficient interpolation and decimation*, IEE Proceedings G 130(6), 1983, pp. 225-235 - doi:10.1049/ip-g-1.1983.0039 (added 2026-10-26, link not checked); the closed-form design as in L. de Soras' public hiir library	polyphase IIR halfbands of the oversampler (12.1)
[26]	S. P. Lipshitz, R. A. Wannamaker, J. Vanderkooy, *Quantization and Dither: A Theoretical Survey*, J. Audio Eng. Soc. 40(5), 1992, pp. 355-375 (added 2026-10-26, link not checked)	TPDF dither (14.1a)
[27]	D. Lemire, *Streaming Maximum-Minimum Filter Using No More than Three Comparisons per Element*, Nordic Journal of Computing 13(4), 2006 - arXiv:cs/0610046 (added 2026-10-28, link not checked)	the limiter's sliding minimum (13)
[28]	F. Adriaensen, *Using a DLL to filter time*, Linux Audio Conference 2005 (added 2026-10-28, link not checked)	the drift loop (17)

Besides these sources the design draws on the project's own measurements, recorded with each decision in `docs/design.md`. Its design notes also refer to the developer's own audio-processing handbook (not public): a knowledge base built from the public standards and papers above and from measurements, which rules out taking presets, values or layouts from commercial products.

Disclaimer. This manual and DJAP are provided "as is", without warranty of any kind (GNU GPL, sections 15 and 16); the measurements describe the builds and test signals named, not every system or piece of music. DJAP's meters are not a certified measuring instrument. The authors are not liable for any damage from using the program or this information. DJAP is not affiliated with, endorsed or sponsored by any company, product or author named here; such names identify sources, standards and the systems DJAP works with.

<img src="images/vst-compatible-logo.png" width="90" alt="VST compatible"> <img src="images/asio-compatible-logo.png" width="90" alt="ASIO compatible">

VST is a registered trademark of Steinberg Media Technologies GmbH. ASIO is a registered trademark of Steinberg Media Technologies GmbH. Windows is a trademark of the Microsoft group of companies. Other names are trademarks of their respective owners.